Singapore Management University

Institutional Knowledge at Singapore Management University

Research Collection School Of Computing and Information Systems | School of Computing and Information Systems

10-2022

CVFNet: Real-time 3D object detection by learning cross view features

Jiaqi GU

Zhiyu XIANG

Pan ZHAO

Tingming BAI

Lingxuan WANG

See next page for additional authors

Follow this and additional works at: https://ink.library.smu.edu.sg/sis_research

 Part of the Artificial Intelligence and Robotics Commons, and the Graphics and Human Computer Interfaces Commons

Citation

GU, Jiaqi; XIANG, Zhiyu; ZHAO, Pan; BAI, Tingming; WANG, Lingxuan; ZHAO, Xijun; and ZHANG, Zhiyuan. CVFNet: Real-time 3D object detection by learning cross view features. (2022). *Proceedings of the 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Kyoto, Japan, October 23-27*. 568-574.

Available at: https://ink.library.smu.edu.sg/sis_research/7945

This Conference Proceeding Article is brought to you for free and open access by the School of Computing and Information Systems at Institutional Knowledge at Singapore Management University. It has been accepted for inclusion in Research Collection School Of Computing and Information Systems by an authorized administrator of Institutional Knowledge at Singapore Management University. For more information, please email cherylds@smu.edu.sg.

Author

Jiaqi GU, Zhiyu XIANG, Pan ZHAO, Tingming BAI, Lingxuan WANG, Xijun ZHAO, and Zhiyuan ZHANG

This conference proceeding article is available at Institutional Knowledge at Singapore Management University: https://ink.library.smu.edu.sg/sis_research/7945

arXiv:2203.06585v2 [cs.CV] 15 Jul 2022

CVFNet: Real-time 3D Object Detection by Learning Cross View Features

Jiaqi Gu[1], Zhiyu Xiang[2], Pan Zhao[1], Tingming Bai[1], Lingxuan Wang[1], Xijun Zhao[3], Zhiyuan Zhang[2,4]

***Abstract*— In recent years 3D object detection from LiDAR point clouds has made great progress thanks to the development of deep learning technologies. Although voxel or point based methods are popular in 3D object detection, they usually involve time-consuming operations such as 3D convolutions on voxels or ball query among points, making the resulting network inappropriate for time critical applications. On the other hand, 2D view-based methods feature high computing efficiency while usually obtaining inferior performance than the voxel or point based methods. In this work, we present a real-time view-based single stage 3D object detector, namely CVFNet to fulfill this task. To strengthen the cross-view feature learning under the condition of demanding efficiency, our framework extracts the features of different views and fuses them in an efficient progressive way. We first propose a novel Point-Range feature fusion module that deeply integrates point and range view features in multiple stages. Then, a special Slice Pillar is designed to well maintain the 3D geometry when transforming the obtained deep point-view features into bird's eye view. To better balance the ratio of samples, a sparse pillar detection head is presented to focus the detection on the nonempty grids. We conduct experiments on the popular KITTI and NuScenes benchmark, and state-of-the-art performances are achieved in terms of both accuracy and speed.**

I. Introduction

3D object detection, which outputs a series of 3D bounding boxes specifying the size, 3D pose and class of the objects in the environment, is an essential perception task for autonomous vehicle. In recent years, LiDAR has become one of the most widely used sensors for autonomous vehicles because of its accurate distance measurement and the robustness to illumination changes. It therefore becomes imperative to design effective 3D object detectors to boost the autonomous navigation. However, efficiently detecting objects in 3D LiDAR point clouds still remains challenging due to the irregular format and the sparsity of the point cloud.

[1]Jiaqi Gu, Pan Zhao, Tingming Bai and Lingxuan Wang are with College of Information Science & Electronic Engineering, Zhejiang University, Hangzhou, China, {vadin, pan_zhao, incredibai, 21960188}@zju.edu.cn

[2]Zhiyu Xiang, corresponding author, is with Zhejiang Provincial Key Laboratory of Information Processing, Communication and Networking, Zhejiang University, Hangzhou, China.xiangzy@zju.edu.cn

[3]Xijun Zhao, is with China North Vehicle Research Institute, Beijing, china.heejunzhao@163.com

[4]Zhiyuan Zhang, is with Ningbo Research Institute, Zhejiang University, Ningbo, China.cszyzhang@gmail.com

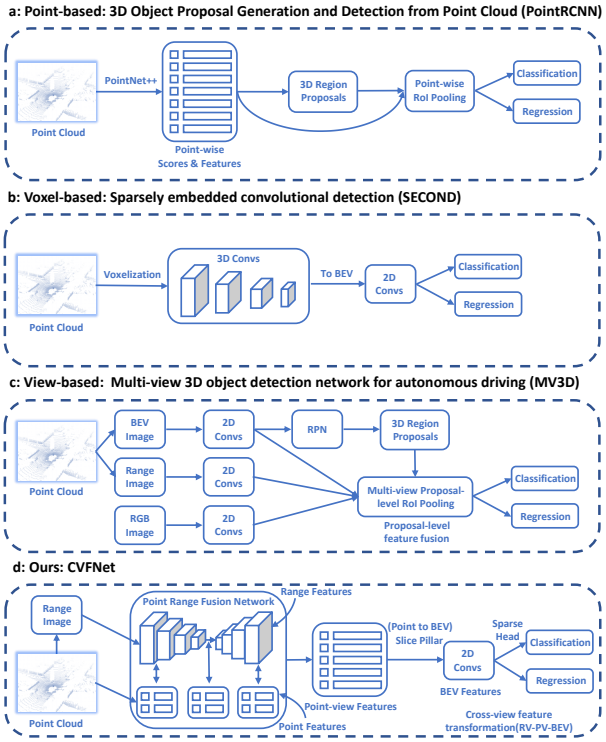


Fig. 1. Comparison with state-of-the-art 3D object detection methods with different representations. Instead of using time-consuming PointNet++ or 3D Convolutions in (a), (b) and proposal-level feature aggregation in (c), our method rely on 2D convolutions and fuse the cross-view features in an effectively progressive way.

As Convolutional Neural Networks (CNN) have shown great advances on 2D object detection, some works try to adapt the success of CNN to 3D object detection by dividing the point cloud into voxels followed by 3D convolutions to learn the voxel-wise features [1]–[5]. Since LiDAR point clouds are sparse by nature, most computation is wasted on the redundant empty voxels, which makes these methods inefficient [1], [3], [6]. Although sparse convolutions [7], [8] can be employed for acceleration [4], the efficiency and deployment are still inferior to the 2D counterpart [9], [10].

Inspired by the PointNet [11] and its variant PointNet++ [12] which pioneer the point-wise feature extraction, point-based 3D detection methods [13]–[16] are proposed to directly estimate the 3D bounding boxes from raw point cloud. While it is free of quantization loss compared to the voxel-based methods, the ball query operations for aggregating local features from neighboring points involved in PointNet++ are time-consuming.

Another line are view-based works which project the

whole point cloud to some 2D views (*e.g*. bird's eye view (**BEV**) [17], [18] or range view [19], [20]) and rely on 2D convolutions to extract features. They usually extract features from a single view image or concatenate different view features together to accomplish the detection. These methods attract increasing attention in recent years due to its high computing efficiency. However, view-based approaches usually suffer from the information loss during the projection from 3D points to 2D views, which is hard to be compensated during 3D detection. When multiple views are employed, directly concatenating features in proposal level as in MV3D [17] can weaken the consistency among features. These shortcomings lead to inferior performance of the view-based methods than the voxel or point based counterparts.

In this work, we propose a novel real-time single stage 3D object detector, namely CVFNet, which is able to learn and fuse cross-view features more consistently. In addition to common range view and bird's eye view, a simple point view which only concerns with MLP operations is induced in our network to reduce the information loss. We transform views and fuse features in a progressive manner to relief feature inconsistency [21]. We first propose a Point-Range fusion module (PRNet) to extract and fuse point-view and range-view features with efficient feature interaction. By this means, we obtain 3D point-wise features with well-preserved 3D geometric and 2D textural information. Next, we design a Slice Pillar transformation module to transform the point-view features into BEV features. To further improve the performance and save the computational cost, a Sparse Pillar Head is utilized to filter out the empty pillars. We conduct experiments on the KITTI [22] and NuScenes [23] dataset and state-of-the-art performances are achieved among the view-based methods. Meanwhile, our framework features utilizing only 2D convolutions (including MLPs), leading to high real-time performance close to PointPillars [24].

The main contributions can be summarized as follows:

- A novel real-time view-based one stage 3D object detector is proposed. It comprises only 2D convolutions and is able to learn consistent cross-view features through a progressive fusion manner.
- A Point-Range fusion module PRNet is designed to effectively extract and fuse the 3D point and 2D range view features in a bilateral aggregation way.
- A Slice Pillar transformation module is presented to transform 3D point features into 2D BEV features, which features less 3D geometric information loss during the dimension collapse.
- Our proposed CVFNet model outperforms existing view-based detectors on KITTI and NuScenes dataset and also shows competitive performance to point or voxel based methods.

II. Related Work

This section reviews the 3D object detection approaches according to the base representations of 3D point clouds. The methods can be roughly classified into three categories: point-based, voxel-based and view-based methods.

Point-based 3D Object Detector. The pioneering work PointNet [11] and PointNet++ [12] which extract point-wise feature directly from point cloud also inspire a lot of 3D object detection methods [13]–[15], [25], [26]. F-PointNet [13] employs 2D detectors to narrow the search space of points to a frustum and regresses 3D bounding box within it. PointRCNN [14] leverages PointNet++ [12] with the set abstraction layer and feature propagation layer to segment foreground points from raw point clouds to reduce the searching space, and predicts 3D bounding boxes for each instance point. Point-based methods are relatively slow due to the ball query operation in feature propagation layer. Instead of using the propagation layers, we propose PR-fusion blocks to aggregate local features and expand the receptive field for point-wise features.

Voxel-based 3D Object Detector. Voxel-based methods divide the point cloud into voxels and rely on 3D convolutions for feature learning. In VoxelNet [3], points are equally grouped into voxels and passed to feature encoding layers to predict 3D boxes. SECOND [4] designs an effective sparse convolution algorithm by encoding valid voxels only, which improves the running speed significantly. Voxel R-CNN [27] takes full advantage of voxel features in a two-stage approach and verifies that coarse voxel granularity can also offer sufficient accuracy. These voxel-based methods are generally effective for accurate 3D object detection, while at the cost of using computation-intensive 3D convolution.

View-based 3D Object Detector. This category includes bird's eye view (BEV) based and range view based methods. BEV-based methods attract much attention since it features no scale variation for scenes. Pixor [18] proposes a novel loss function for 3D object detection on BEV map. PointPillars [24] directly applies PointNet [11] to collapse the height dimension and use it as a pseudo-image for detection. Range view is another popular representation with minor information loss, which is consistent with the point production pattern of rotated LiDAR sensors. LaserNet [28] exploits the range-view in their own dataset and predicts a multimodal distribution for each point. There are also methods integrating multiple views to improve the detection performance. MV3D [17] fuses the features from BEV, range view as well as RGB images. AVOD [29] expands MV3D by exploiting the 3D anchors into BEV and RGB images. These methods mainly extract view features independently and aggregate them at proposal stage, which may cause inconsistency among features. In contrast, our method learns cross-view features in a progressive way, *i*,*.e*., transforming features in sequential views so that the feature inconsistency problem can be largely relieved.

III. Method

In this section, we describe our CVFNet model in detail. As illustrated in Fig 2, the framework mainly comprises the Point-Range fusion module, Slice Pillar transformation, and Sparse Pillar Head. Given the input of point and range view, the features are learned and aggregated effectively through 'Range - Point - BEV' views in a progressive way. Rather

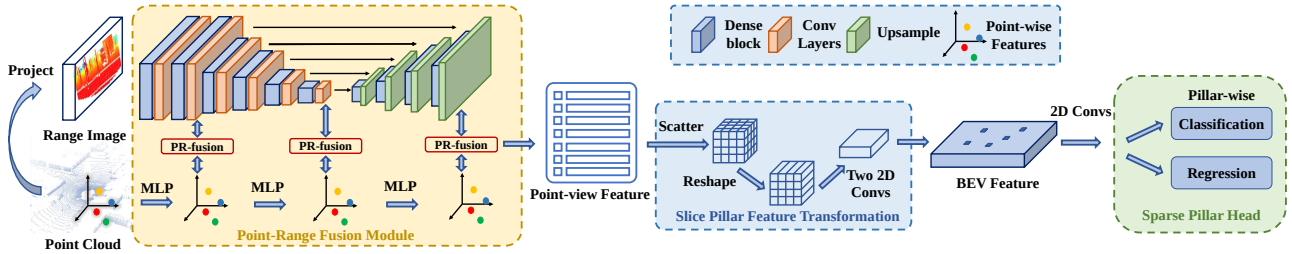


Fig. 2. Illustration of the architecture of CVFNet. The features are learned and transformed progressively in 'Range – Point – BEV' order along which the Point view serves as reference.

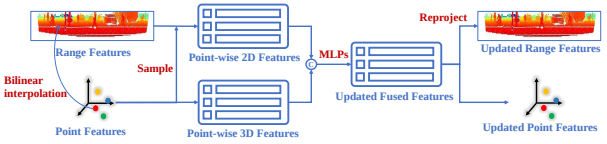


Fig. 3. Our PR-fusion block to update both Point and Range features.

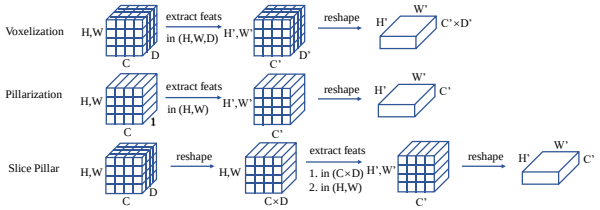


Fig. 4. The Comparison of Point-to-BEV Feature Transformation Modules.

than 3D convolutions or time-consuming ball query point operations, our model involves only fast 2D convolutions and MLPs, which guarantee the real-time performance of the system.

A. Point-Range Fusion Module

Range view is the native form of rotating LiDAR, featuring dense range images distributed in spherical coordinates where 2D convolutions can be applied effectively. Point view is of 3D coordinates that multi-layer perceptron (MLPs) can be used to extract features for each point. To extract and aggregate the features of these two views, we propose a Point-Range fusion module within which multi-stage bilateral fusion is carried out.

Point-Range Mapping. Given a 3D point cloud as input, we first transform it into range representation as in [30]. Each point $\mathbf{p}_i = (x, y, z)$ is converted via a mapping $\Pi : \mathbb{R}_3 \mapsto \mathbb{R}_2$ to spherical coordinates, which are then scaled as planar image coordinates (u, v) within an image of size $h \times w$. The process is described as:

$$\begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} \frac{1}{2}[1 - \arctan(y,x)\pi^{-1}]w \\ [1 - (\arcsin(zr^{-1}) + \text{fov}_{\text{up}})\text{fov}^{-1}]h \end{pmatrix}$$

where (h, w) are the height and width of the desired range image, fov and fov_{up} are the entire and upper part of vertical field-of-view of the sensor respectively, and $r = ||\mathbf{p}_i||_2$ is the range of each point. This procedure produces a point-to-pixel index table, which can help transform the features between point and range view.

Module Structure. As shown in Fig 2, the Point-Range fusion module comprises two main streams, *i.e.*, Point stream and Range stream, interactively connected by three PR-fusion blocks. The Range stream is an encoder-decoder architecture based on 2D denseblocks [31]. In the encoder stage, we insert denseblocks with a stride of 2 to gradually produce 2x, 4x, 8x, 16x downscaled feature maps. While in the decoder stage, we apply denseblocks with a stride of 1, and use bilinear interpolation to gradually recover the original size. To better utilize the encoder features, we directly concatenate the encoder features into the the decoder stage as in U-Net [32]. For the Point stream, we just use several PointNet-like MLPs (a series of fully-connected layers) for feature extraction. We do not use any complex set abstraction layers or feature propagation layers in [12] to avoid time-consuming operations.

PR-fusion Block. The PR-fusion block is responsible for deeply fusing and propagating the features between point and range view. The detailed structure of this block is illustrated in Fig 3. Given the range and point view features in a certain stage of the main stream, we first sample 2D range features for each 3D point by searching the point-range indexing table. To deal with the scale variation, we use bilinear interpolation to get the features at the right scale. And for point-view features, we directly get the point-wise 3D features from the feature map. Then, we concatenate these point-wise 2D and 3D features and pass them to a series of MLPs. The resulting aggregated features are reprojected back to the range and point view respectively to continue the subsequent process of the network. The PR-fusion block is applied three times in the module, corresponding to early, middle and late stage of fusion, respectively.

B. Slice Pillar Transformation

Bird's eye view(BEV) is a popular representation in 3D object detection as it features no occlusion or scale variation. There are two common ways to project the point cloud to BEV: voxelization and pillarization.

Given a voxelized point cloud with the shape of $(\text{H}, \text{W}, \text{D}, \text{C})$, voxel-based methods extract features by 3D convolutions in H, W and D dimensions, where D represents height dimension and C represents feature channels. Downsampled features with the size of $(\text{H}^{'}, \text{W}^{'}, \text{D}^{'}, \text{C}^{'})$ are then reshaped into BEV features with size $(\text{H}^{'}, \text{W}^{'}, \text{D}^{'} \times \text{C}^{'})$. On the other hand, pillar-based methods directly use PointNet to extract features for points in single pillars with size $(\text{H}, \text{W}, 1, \text{C})$, which has no feature resolution on height dimensions. It is then formatted as $(\text{H}^{'}, \text{W}^{'}, \text{C}^{'})$ on the BEV

perpective before feeding it into the detection head. While voxel-based methods maintain smaller feature granularity, they rely on 3D convolutions which are computation intensive. Pillar-based methods [24] are more computationally efficient, while they suffer information loss along the height dimension.

To tackle these problems, we design an improved version of the pillar strategy, namely Slice Pillar. The differences to the previous two transformations are shown in Fig 4. We first calculate the 3D index of each point and use scatter operations, which has a much higher speed than voxel generator, to transform the point-wise features into voxel-wise feature volume with size of $(\mathrm{H}, \mathrm{W}, \mathrm{D}, \mathrm{C})$. Scatter operation is implemented in Pytorch and will pick one point arbitrarily while propagating the gradient to points that have the same voxel index. Then we combine the last two dimensions of features, resulting in a feature volume of $(\mathrm{H}, \mathrm{W}, \mathrm{D} \times \mathrm{C})$. Upon these sliced pillars, a couple of fully-connected layers (or 1×1 convolutions) are applied along $(\mathrm{D} \times \mathrm{C})$ dimension, followed by a series of common 2D convolutions along BEV's planar (H, W) dimensions. By this means, fine granularity features in all three dimensions can be extracted without using 3D convolutions.

C. Sparse Pillar Head

After the slice pillar transformation module and a series of convolution layers on BEV, we have the BEV feature map at hand. To estimate the proposals in each BEV grid, most existing methods use a dense prediction head. However, since most grids are empty, we introduce a sparse pillar head to suppress the useless proposals. The sparse pillar head consists of classification and regression head. Given a sparse BEV map, we first define nonempty grids as valid pillars and use 2D convolutions to extract features, which are then indexed to the pillar-wise features. Finally, classification head is used to predict the probability of a pillar's class, and regression head is responsible for regressing 3D box parameters. The advantages of our sparse pillar head are two folds. Firstly, it balances the distribution of positive and negative samples by filtering out the empty grids which are otherwise counted as negative samples. Secondly, the computation cost is further reduced.

D. Loss Function

The overall loss for KITTI dataset consists of pillar-wise anchor classification loss $\mathrm{L_{cls}}$, regression loss $\mathrm{L_{reg}}$ and direction classification loss $\mathrm{L_{dir}}$. The final loss is the weighted sum of three losses:

$$\mathrm{Loss} = \mathrm{L_{cls}} + \alpha \mathrm{L_{reg}} + \beta \mathrm{L_{dir}}$$

where α is set to 2.0 and β is set to 0.2. We use the focal loss with default parameters as [43] for $\mathrm{L_{cls}}$ and $\mathrm{L_{dir}}$. For $\mathrm{L_{reg}}$, the smooth-L1 loss is employed to calculate the residual value relative to the predefined anchor of ground truth boxes.

In NuScenes dataset, we use center-based anchor-free head to be adaptive to the requirement of 10 categories detection. The loss follows CenterPoint [44] as:

$$\mathrm{Loss} = \mathrm{L_{hm}} + \lambda_{\mathrm{box}} \mathrm{L_{box}} + \lambda_{\mathrm{local}} \mathrm{L_{local}}$$

where $\mathrm{L_{hm}}$ is the loss of heatmap for center classification, $\mathrm{L_{box}}$ is the loss for regressing box parameters, $\mathrm{L_{local}}$ is the loss for regressing local offset and λs are the related loss weights, which are both set to 0.25.

IV. Experiments

We conduct experiments on the KITTI and NuScenes dataset. We describe the dataset settings in Sec IV-A and implementation details in Sec IV-B. In Sec IV-C, we compare our CVFNet with the state-of-the-art 3D object detection methods. Ablation studies are conducted in Sec IV-D.

A. Dataset and Evaluation

We compare the quantitative performance of our method on well-known KITTI and NuScenes dataset.

The KITTI [22] 3D object detection dataset contains 7481 training samples and 7518 test samples. The training set is divided into 3712 samples for training and 3769 samples for validation [45]. The samples are classified as Car, Pedestrian and Cyclist with three difficulty levels for each class: Easy, Moderate, and Hard. The official KITTI leaderboard is ranked on Moderate difficulty. CVFNet is evaluated on the KITTI *test* set by submitting the detection results to the official server. We use average precision (AP) with an IoU threshold 0.7 for Car and 0.5 for both Pedestrian and Cyclist.

The NuScenes [23] 3D detection dataset contains 1000 scenes, including 700 scenes for training, 150 scenes for validation and 150 scenes for testing. Each scene is of 20*s* duration and captured by a 32-beam LiDAR. There are up to 1.4M annotated 3D bounding boxes for 10 classes: car, truck, bus, trailer, construction vehicle, pedestrian, bicycle, barrier and traffic cone. The mAP is for measuring the precision and recall which is defined based on the match by 2D center distance on the ground plane instead of IoU. NDS (Nuscenes Detection Score) is the composite score in terms of the precision of translation, scale, orientation, velocity and attributes.

B. Implementation Details

For Point-Range fusion module, we project the raw point cloud into range image with the size of $48 \times 512 \times 5$ as [19] in KITTI and $64 \times 2048 \times 6$ in NuScenes dataset, respectively. The input five channels for each pixel include range, 3D x, y, z coordinates and intensity. An extra timestamp channel is added in NuScenes dataset as multi-sweep strategy is applied. The backbone for range stream is an encoder-decoder structure with 10 denseblocks. The backbone network extracts range features with strides of (1,1,2,2,2,2) and 4 denseblocks with bilinear interpolation operations to gradually recover the original size. We insert 6 convolution layers after each denseblock in encoders with the number of (3,3,5,5,5,5) layers to extract explicit features. For the MLPs in Point stream, we apply a couple of fully connected layers like PointNet [11] to extract 3D point-wise features. For the

TABLE I

COMPARISONS OF 3D OBJECT DETECTION ON KITTI *test* SET. THE BEST SCORES ARE MARKED IN BOLD WITH '*' INDICATES RGB IMAGES INTEGRATED. ALL RUNTIME VALUES ARE COLLECTED FROM KITTI BENCHMARK AS WELL AS OFFICIAL PAPER AND CODE.

Method	Representation	Stage	3D AP			BEV AP			Time(ms)
			Easy	Moderate	Hard	Easy	Moderate	Hard	
F-PointNet* [13]	Point	Multi	82.19	69.79	60.59	91.17	84.67	74.77	170
PI-RCNN* [33]	Point	Two	84.37	74.82	70.03	91.44	85.81	81.00	100
PointRCNN [14]	Point	Two	86.96	75.64	70.70	92.13	87.39	82.72	100
F-ConvNet* [15]	Point	Multi	87.36	76.39	66.69	91.51	85.84	76.11	470
PointGNN [25]	Point	Two	88.33	79.47	72.29	**93.11**	**89.17**	83.90	643
3D-SSD [26]	Point	One	**88.36**	**79.57**	**74.55**	92.66	89.02	**85.86**	**38**
VoxelNet [3]	Voxel	One	77.82	64.17	57.51	87.95	78.39	71.29	220
UberATG-ContFuse* [34]	Voxel	One	83.68	68.78	71.67	94.07	85.35	75.88	60
SECOND [4]	Voxel	One	83.34	72.55	65.82	89.39	83.77	78.59	50
TANet [35]	Voxel	One	84.39	75.94	68.82	91.58	86.54	81.19	**34.75**
3D IoU Loss [36]	Voxel	Two	86.16	76.50	71.39	91.36	86.22	81.20	80
Fast PointRCNN [5]	Voxel	Two	85.29	77.40	70.24	90.87	87.84	80.52	65
Associate-3Ddet [37]	Voxel	One	85.99	77.40	70.53	91.40	88.09	82.96	60
PV-RCNN [38]	Voxel	Two	90.25	81.43	76.82	**94.98**	**90.65**	**86.14**	80
Voxel-RCNN [27]	Voxel	Two	**90.90**	**81.62**	**77.06**	94.85	88.83	86.13	40
MV3D* [17]	View	Two	74.97	63.63	54.00	86.62	78.93	69.80	360
AVOD-FPN* [29]	View	Two	83.07	71.76	65.73	90.99	84.82	79.62	100
UberATG-Pixor++ [18]	View	One	-	-	-	93.28	86.01	80.11	35
MoDet [39]	View	One	-	-	-	90.80	87.56	82.69	50
PointPillars [24]	View	One	82.58	74.31	68.99	90.07	86.56	**82.81**	**23.6**
LaserNet [28]	View	One	-	-	-	79.19	74.52	68.45	30
MAFF-Net* [40]	View	One	85.52	75.54	67.61	90.79	87.34	77.66	40
SARPNET [41]	View	One	85.63	76.64	71.31	92.21	86.92	81.68	50
Ours	View	One	**88.75**	**77.70**	**71.95**	**93.65**	**87.87**	82.29	**28.1**

TABLE II

PERFORMANCE COMPARISION OF AP_{3D} IN OTHER CATEGORIES ON THE KITTI *validation* SET.

Method	Pedestrian			Cyclist		
	Eas.	Mod.	Har.	Eas.	Mod.	Har.
PointPillars [24]	54.18	49.70	45.00	80.38	63.29	60.05
PFFNet [42]	55.46	50.09	45.49	-	-	-
Ours	**60.79**	**53.95**	**49.04**	**83.07**	**65.07**	**61.34**

detection head, we use an anchor-based head [24] in KITTI dataset and an anchor-free head [46] in NuScenes dataset.

For the scatter operations in KITTI dataset, we define the detection range as [0, 69.12]m for X axis, [-39.68, 39.68]m for Y axis and [-3, 1]m for Z axis. We set the scatter voxel size to (0.16, 0.16, 0.2) and the final 3D volumes is of size with $H = 496, W = 432, D = 20$ and $C = 64$, respectively. In Nuscenes dataset, the detection range is $[-51.2, 51.2]$m for the X,Y axis and [-3, 5]m for the Z axis. The voxel size is set to (0.2, 0.2, 0.4) and the final 3D volumes is of size with $H = 512, W = 512, D = 20$ and $C = 64$. After Point-to-BEV transformation, we use three 2D convolution blocks with stride of 2 to downscale features and then upscale each output to $\frac{1}{2}$ of the original size, The maps are finally concatenated together to get the BEV features for the detection head.

We train our network end-to-end with the ADAM optimizer of a batch size 12/8 on two NVIDIA RTX2080Ti GPUs for 80/20 epochs on KITTI/NuScenes dataset, respectively. The learning scheme follows cyclic policy, and the learning rate changes from 0.001 to 0.01, then goes to 0.001×10^{-4}. The momentum ranges from 0.85 to 0.95.

For data augmentation, we use the random flipping along the x axis, random global scaling with the scaling factor in $[0.95, 1.05]$ and random global rotation ranges from $[-\frac{\pi}{4}, \frac{\pi}{4}]$ around the z axis. Some objects are also randomly sampled from the training data and injected into training samples.

C. Comparison with State-of-the-Arts

KITTI We evaluate our CVFNet detector on the KITTI *test* set and submit the detecting results to the official online benchmark server. The mAP performance calculated with 40 recall positions on car category is recorded in Table I. In the context of view-based detectors, our method outperforms all of its competitors by a noticeable margin. In particular, the 3D AP of our model outperforms the baseline method vanilla PointPillars [24] by 6.17%, 3.39% and 1.96% in three difficulty levels respectively. With similar pillar representations on the BEV, this large improvement mainly comes from our specially designed cross view feature learning scheme. Besides, our model even outperforms most of the point-based and voxel-based methods, which verifies the benefits of exploiting 2D view features from 3D point clouds.

We also show the results in three class on KITTI *validation* set in Table II. Results are evaluated by the mAP with 11 recall positions. Compared with the baseline method PointPillars [24], our method achieves about 4.25% and 1.78% improvement on moderate AP for Pedestrian and Cyclist respectively, which further depicts the superior generalization of our method. Some qualitative results are illustrated in Fig 5, where the detected 3D boxes are also projected to the RGB images for better visualization.

NuScenes To test the robustness of our CVFNet, we also evaluate our method on the NuScenes dataset and submit the detection results to the official server. The mAP and NDS performance of 10 categories is listed in Table III. Compared with other view-based methods, our method get

TABLE III

COMPARISONS OF 3D OBJECT DETECTION ON NUSCENES *test* SET.

Method	mAP	NDS	Car	Truck	Bus	Trailer	Cons.	Pedes.	MCycle	bicycle	cone	barrier
PointPillars [24]	30.5	45.3	68.4	23.0	28.2	23.4	4.1	59.7	27.4	1.1	30.8	38.9
SARPNET [41]	31.6	49.7	59.9	18.7	19.4	18.0	11.6	69.4	29.8	14.2	44.6	38.3
CBGS [47]	52.8	**63.3**	81.1	**48.5**	54.9	42.9	10.5	80.1	51.5	22.3	70.9	65.7
3D-CVF [48]	52.7	62.3	83.0	45.0	48.8	49.6	**15.9**	74.2	51.2	30.4	62.9	65.9
PanoNet3D [49]	54.5	63.1	80.1	45.4	54.0	51.7	15.1	79.1	**53.1**	**31.1**	71.9	62.9
CVFNet	**54.9**	**63.3**	**83.0**	47.9	**56.3**	**53.0**	13.7	**80.4**	46.7	27.5	**72.4**	**67.8**

TABLE IV

EFFECTS OF PROPOSED COMPONENTS FOR 'CAR' DETECTION ON KITTI *validation* SET.

Baseline	PRNet	Slice Pillar	Sparse Head	3D			BEV		
				Easy	Moderate	Hard	Easy	Moderate	Hard
✓				87.66	76.01	73.38	92.62	87.38	83.89
✓	✓			88.99	78.64	76.19	93.16	88.42	85.43
✓		✓		89.72	79.37	75.44	93.67	**88.98**	86.31
✓			✓	87.62	77.13	74.77	92.91	88.30	83.98
✓	✓	✓		89.91	79.49	76.42	93.84	88.90	**86.62**
✓	✓	✓	✓	**90.02**	**79.82**	**76.84**	**93.89**	88.96	86.50

TABLE V

EFFECTS OF PROPOSED COMPONENTS ON NUSCENES *validation* SET.

Baseline	PRNet	Slice Pillar	Sparse Head	mAP	NDS
✓				49.60	60.33
✓	✓			51.89	61.48
✓		✓		51.13	60.95
✓			✓	49.99	60.19
✓	✓	✓		53.21	61.97
✓	✓	✓	✓	**53.46**	**62.09**

TABLE VI

EFFECTS OF DIFFERENT VIEW FEATURES

Point	Range	3D			BEV		
		Easy	Moderate	Hard	Easy	Moderate	Hard
✓		87.68	77.55	73.34	92.59	88.54	84.07
	✓	89.11	78.98	75.27	92.62	88.46	85.49
✓	✓	90.02	79.82	76.84	93.89	88.96	86.50

the highest AP in most categories, which indicates our method's robustness for different scenes and objects.

Timing Owing to the fast 2D CNN backbone and one-stage architecture, our CVFNet achieves 35.6 Hz in KITTI and 16.4 Hz in NuScenes when evaluated on a single 2080Ti GPU without any C++/CUDA/TensorRT optimizations. The comparisons of the running speed in KITTI are shown on the rightest column of Table I, where our algorithm ranks the second fastest.

D. Ablation Studies

We conduct ablation studies on both KITTI and NuScenes datasets.

Effects of Components We study the effects of the different components of CVFNet by setting the the vanilla PointPillars [24] as the baseline. The results on KITTI dataset are shown in Table IV. With the PRNet on (Row 2), the 3D and BEV AP has a 1%-3% improvement. Adding the Slice Pillar and Sparse Head alone also have positive effects on the baseline. Adding all of the three components can achieve the best results. The ablation results on Nuscenes shown in Table V also show the similar effects.

Effects of different view features We analyze the effectiveness of fusing point and range views within the Point-Range fusion module. The experimental results on KITTI dataset are shown in Table VI. We can see that the range view alone performs better than the point view. The combination of both views improves the 3D AP by more than 2%, which indicates the success of our point and range view fusion strategy.

V. CONCLUSION

In this paper, we present a cross-view learning based single-stage 3D object detection method which features both high detection and real-time performance. To better preserve 3D geometric information while maintaining computing efficiency, we design a progressive cross view fusion pipeline to learn and transform the features among Point, Range and BEV views. The special designs of the point-range fusion and the slice pillar module strengthen the solid feature learning. Experimental results achieve state-of-the-art performance among the view-based methods with a high frame rate of 35.6Hz on the KITTI dataset. The simple backbone and quick runtime of our CVFNet make it easy to deploy in resource constrained devices. In future work, we plan to integrate more views sun as RGB image to further improve the performance.

REFERENCES

[1] D. Z. Wang and I. Posner, "Voting for voting in online point cloud object detection," in *RSS*, 2015, pp. 10–15 607.

[2] S. Song and J. Xiao, "Deep sliding shapes for amodal 3d object detection in rgb-d images," in *CVPR*, 2016, pp. 808–816.

[3] Y. Zhou and O. Tuzel, "Voxelnet: End-to-end learning for point cloud based 3d object detection," *CVPR*, pp. 4490–4499, 2018.

[4] Y. Yan, Y. Mao, and B. Li, "Second: Sparsely embedded convolutional detection," *Sensors (Basel, Switzerland)*, vol. 18, 2018.

[5] Y. Chen, S. Liu, X. Shen, and J. Jia, "Fast point r-cnn," *ICCV*, pp. 9774–9783, 2019.

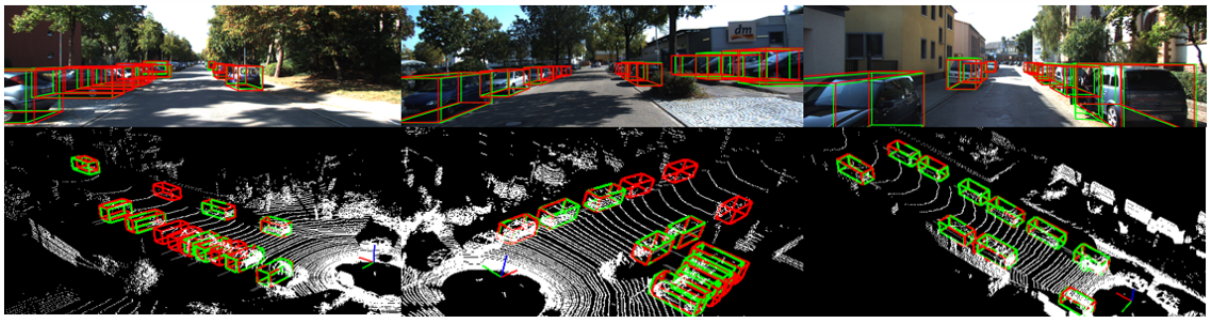


Fig. 5. The detection results on KITTI *validation* set. We render the predicted and ground truth bounding boxes in red and green, respectively.

[6] B. Li, T. Zhang, and T. Xia, "Vehicle detection from 3d lidar using fully convolutional network," in *RSS*, 2016.

[7] B. Graham and L. van der Maaten, "Submanifold sparse convolutional networks," *ArXiv*, 2017.

[8] B. Graham, M. Engelcke, and L. Van Der Maaten, "3d semantic segmentation with submanifold sparse convolutional networks," in *CVPR*, 2018, pp. 9224–9232.

[9] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," in *ICCV*, 2017, pp. 2961–2969.

[10] J. Redmon and A. Farhadi, "Yolo9000: better, faster, stronger," in *CVPR*, 2017, pp. 7263–7271.

[11] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *CVPR*, July 2017.

[12] C. Qi, L. Yi, H. Su, and L. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," in *NeurIPS*, 2017.

[13] C. Qi, W. Liu, C. Wu, H. Su, and L. Guibas, "Frustum pointnets for 3d object detection from rgb-d data," *CVPR*, pp. 918–927, 2018.

[14] S. Shi, X. Wang, and H. Li, "Pointrcnn: 3d object proposal generation and detection from point cloud," *CVPR*, pp. 770–779, 2019.

[15] Z. Wang and K. Jia, "Frustum convnet: Sliding frustums to aggregate local point-wise features for amodal," *IROS*, pp. 1742–1749, 2019.

[16] Z. Yang, Y. Sun, S. Liu, X. Shen, and J. Jia, "Std: Sparse-to-dense 3d object detector for point cloud," in *ICCV*, 2019, pp. 1951–1960.

[17] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, "Multi-view 3d object detection network for autonomous driving," *CVPR*, pp. 6526–6534, 2017.

[18] B. Yang, W. Luo, and R. Urtasun, "Pixor: Real-time 3d object detection from point clouds," *CVPR*, pp. 7652–7660, 2018.

[19] Z. Liang, M. Zhang, Z. Zhang, X. Zhao, and S. Pu, "Rangercnn: Towards fast and accurate 3d object detection with range image representation," *ArXiv*, 2020.

[20] Z. Liang, Z. Zhang, M. Zhang, X. Zhao, and S. Pu, "Rangeioudet: Range image based real-time 3d object detector optimized by intersection over union," in *CVPR*, 2021, pp. 7140–7149.

[21] C. Wang, C. Ma, M. Zhu, and X. Yang, "Pointaugmenting: Cross-modal augmentation for 3d object detection," in *CVPR*, 2021, pp. 11 794–11 803.

[22] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *The International Journal of Robotics Research*, vol. 32, pp. 1231 – 1237, 2013.

[23] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuscenes: A multimodal dataset for autonomous driving," *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11 618–11 628, 2020.

[24] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "Pointpillars: Fast encoders for object detection from point clouds," in *CVPR*, June 2019.

[25] W. Shi and R. Rajkumar, "Point-gnn: Graph neural network for 3d object detection in a point cloud," *CVPR*, pp. 1708–1716, 2020.

[26] Q. Luo, H. Ma, L. Tang, Y. Wang, and R. Xiong, "3d-ssd: Learning hierarchical features from rgb-d images for amodal 3d object detection," *Neurocomputing*, vol. 378, pp. 364–374, 2020.

[27] J. Deng, S. Shi, P. Li, W. Zhou, Y. Zhang, and H. Li, "Voxel r-cnn: Towards high performance voxel-based 3d object detection," in *AAAI*, 2021.

[28] G. P. Meyer, A. Laddha, E. Kee, C. Vallespi-Gonzalez, and C. K. Wellington, "Lasernet: An efficient probabilistic 3d object detector for autonomous driving," *CVPR*, pp. 12 669–12 678, 2019.

[29] J. Ku, M. Mozifian, J. Lee, A. Harakeh, and S. L. Waslander, "Joint 3d proposal generation and object detection from view aggregation," *IROS*, pp. 1–8, 2018.

[30] A. Milioto, I. Vizzo, J. Behley, and C. Stachniss, "Rangenet++: Fast and accurate lidar semantic segmentation," *IROS*, pp. 4213–4220, 2019.

[31] G. Huang, Z. Liu, and K. Q. Weinberger, "Densely connected convolutional networks," *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2261–2269, 2017.

[32] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *MICCAI*, 2015.

[33] L. Xie, C. Xiang, Z. Yu, G. Xu, Z. Yang, D. Cai, and X. He, "Pi-rcnn: An efficient multi-sensor 3d object detector with point-based attentive cont-conv fusion module," in *AAAI*, 2020, pp. 12 460–12 467.

[34] M. Liang, B. Yang, S. Wang, and R. Urtasun, "Deep continuous fusion for multi-sensor 3d object detection," in *ECCV*, 2018.

[35] Z. Liu, X. Zhao, T. Huang, R. Hu, Y. Zhou, and X. Bai, "Tanet: Robust 3d object detection from point clouds with triple attention," in *AAAI*, 2020, pp. 11 677–11 684.

[36] D. Zhou, J. Fang, X. Song, C. Guan, J. Yin, Y. Dai, and R. Yang, "Iou loss for 2d/3d object detection," *3DV*, pp. 85–94, 2019.

[37] L. Du, X. Ye, X. Tan, J. Feng, Z. Xu, E. Ding, and S. Wen, "Associate-3ddet: Perceptual-to-conceptual association for 3d point cloud object detection," *CVPR*, pp. 13 326–13 335, 2020.

[38] S. Shi, C. Guo, L. Jiang, Z. Wang, J. Shi, X. Wang, and H. Li, "Pv-rcnn: Point-voxel feature set abstraction for 3d object detection," *CVPR*, pp. 10 526–10 535, 2020.

[39] Y. Zhang, Z. Xiang, C. Qiao, and S. Chen, "Accurate and real-time object detection based on bird's eye view on 3d point clouds," *3DV*, pp. 214–221, 2019.

[40] Z. Zhang, M. Zhang, Z. Liang, X. Zhao, M. Yang, W. Tan, and S. Pu, "Maff-net: Filter false positive for 3d vehicle detection with multi-modal adaptive feature fusion," *ArXiv*, vol. abs/2009.10945, 2020.

[41] Y. Ye, H. Chen, C. Zhang, X. Hao, and Z. Zhang, "Sarpnet: Shape attention regional proposal network for lidar-based 3d object detection," *Neurocomputing*, vol. 379, pp. 53–63, 2020.

[42] J. Fu, G. Ren, Y. Chen, and S. Liu, "Improved pillar with fine-grained feature for 3d object detection," *ArXiv*, vol. abs/2110.06049, 2021.

[43] T.-Y. Lin, P. Goyal, R. B. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *ICCV*, pp. 2999–3007, 2017.

[44] T. Yin, X. Zhou, and P. Krähenbühl, "Center-based 3d object detection and tracking," *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11 779–11 788, 2021.

[45] X. Chen, K. Kundu, Y. Zhu, A. G. Berneshawi, H. Ma, S. Fidler, and R. Urtasun, "3d object proposals for accurate object class detection," in *NeurIPS*, 2015.

[46] T. Yin, X. Zhou, and P. Krähenbühl, "Center-based 3d object detection and tracking," *CVPR*, 2021.

[47] B. Zhu, Z. Jiang, X. Zhou, Z. Li, and G. Yu, "Class-balanced grouping and sampling for point cloud 3d object detection," *ArXiv*, vol. abs/1908.09492, 2019.

[48] J. H. Yoo, Y. Kim, J. S. Kim, and J. W. Choi, "3d-cvf: Generating joint camera and lidar features using cross-view spatial feature fusion for 3d object detection," in *ECCV*, 2020.

[49] X. Chen, J. Wang, D. Held, and M. Hebert, "Panonet3d: Combining semantic and geometric understanding for lidar point cloud detection," *2020 International Conference on 3D Vision (3DV)*, pp. 753–761, 2020.